

Advance DevOps Practicals

Case Study :

3. Infrastructure as Code with Terraform

- **Concepts Used:** Terraform, AWS S3, and EC2.
- **Problem Statement:** "Use Terraform to provision an AWS EC2 instance and an S3 bucket. Deploy a sample static website on the S3 bucket using the EC2 instance as the backend server."
- **Tasks:**
 - Write a Terraform script to create an EC2 instance and an S3 bucket.
 - Deploy the static website on the S3 bucket.
 - Use the EC2 instance to interact with the S3 bucket and log the actions.

1. Introduction

Case Study Overview:

This case study delves into leveraging **Terraform**, an open-source Infrastructure-as-Code (IaC) tool, to automate the provisioning of AWS resources, specifically **EC2 instances** and **S3 buckets**. Terraform enables the declaration of cloud infrastructure in code, making it easier to automate, scale, and manage cloud environments. By using configuration files, it allows infrastructure to be provisioned, modified, and destroyed in a repeatable and version-controlled manner.

In this particular case study, we focus on the following components:

- **Amazon EC2 (Elastic Compute Cloud):** These are virtual servers in the cloud, providing scalable computing capacity. EC2 allows users to launch instances (virtual machines) with specific operating systems and configurations.
- **Amazon S3 (Simple Storage Service):** A storage service designed for scalable object storage. S3 is commonly used for storing and retrieving large amounts of data, backups, and content delivery.

The purpose of this case study is to showcase how Terraform automates the creation of both EC2 instances for compute resources and S3 buckets for storage requirements. This process is managed through concise and reusable configuration files written in HashiCorp Configuration Language (HCL), eliminating the need for manual intervention via AWS Management Console or CLI. This automation not only speeds up infrastructure provisioning but also minimizes the risks of human error, making it ideal for production environments where consistency and scalability are critical.

This study also explores the use of Terraform modules and state management, allowing for efficient collaboration, infrastructure scaling, and resource management. By utilizing Terraform's declarative approach, infrastructure becomes easier to manage and can be updated or replicated in different environments with minimal effort.

Key Feature and Application:

The **key feature** of this case study is the automated provisioning of cloud infrastructure using Terraform. By writing Terraform configuration files, users can define the infrastructure they need, such as the number and type of EC2 instances, security groups, VPCs (Virtual Private Clouds), and S3 buckets. When the configuration is applied, Terraform automatically provisions these resources in AWS.

Some practical applications of this feature include:

1. Automated EC2 Instance Deployment:

- EC2 instances are automatically created based on the specifications in the configuration file, such as the instance type (e.g., t2.micro), operating system, security groups, and more. This automation eliminates the need to manually launch instances through the AWS console or CLI, greatly reducing setup time.
- Terraform also enables scalable infrastructure by defining autoscaling groups, ensuring that the infrastructure can dynamically adapt to changing workloads by adding or removing instances as needed.

- Practical application: This feature is ideal for setting up development, testing, or production environments in a consistent and repeatable manner, allowing DevOps teams to spin up environments on demand.

2. Automated S3 Bucket Provisioning:

- Terraform automates the creation of S3 buckets, where you can specify bucket names, versioning, lifecycle rules, and access policies. This ensures that storage is always available as per the configuration without manual intervention.
- S3 buckets can also be linked with other services such as Lambda for serverless computing, which further extends the automation capabilities.
- Practical application: Automating S3 bucket provisioning is useful for managing large-scale backups, log storage, data archiving, or delivering static website content. It guarantees that storage is consistently set up as required across multiple environments.

3. State Management:

- Terraform keeps track of the state of your infrastructure, allowing it to detect changes and apply only the necessary updates (referred to as “incremental provisioning”). This ensures that infrastructure updates are efficiently managed without needing to re-provision all resources from scratch.
- Practical application: In production environments where downtime must be minimized, Terraform’s ability to apply only incremental updates allows for safe and controlled changes.

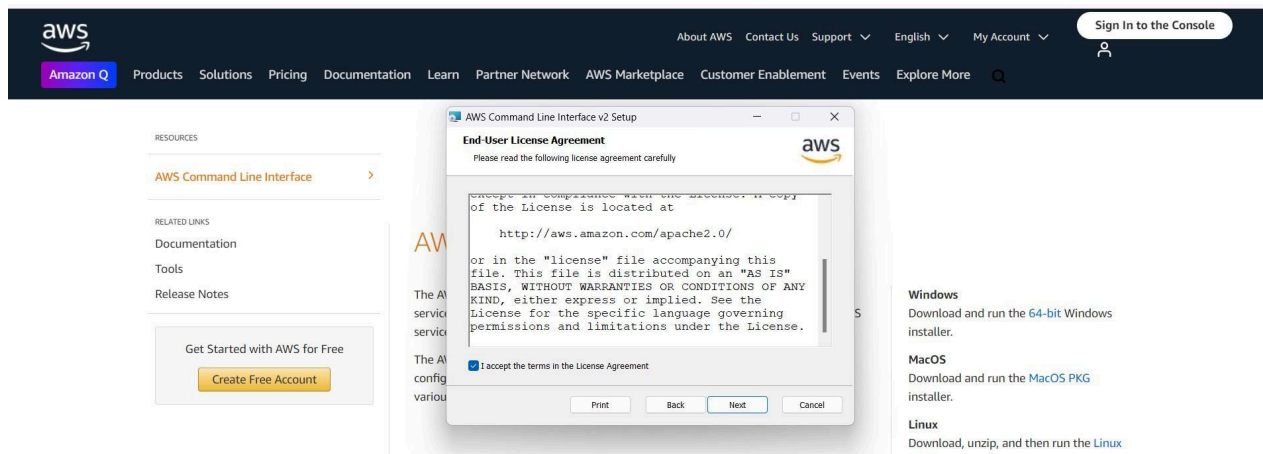
4. Version Control and Reusability:

- Terraform configurations can be version-controlled using Git or other tools, making it easy to track changes and collaborate with other team members. Terraform modules can also be reused across different projects, speeding up infrastructure provisioning and promoting consistency.
- Practical application: For large-scale organizations with multiple teams working in parallel, Terraform allows for centralized management of infrastructure code, improving collaboration and reducing the risk of configuration drift.

The **application** of this automation is particularly useful in scenarios where cloud infrastructure needs to be deployed quickly, consistently, and at scale. By automating EC2 and S3 resource creation, DevOps teams can ensure that environments are set up exactly as defined, allowing them to focus on higher-level tasks like application development and performance optimization instead of infrastructure management.

2. Step-by-Step Explanation

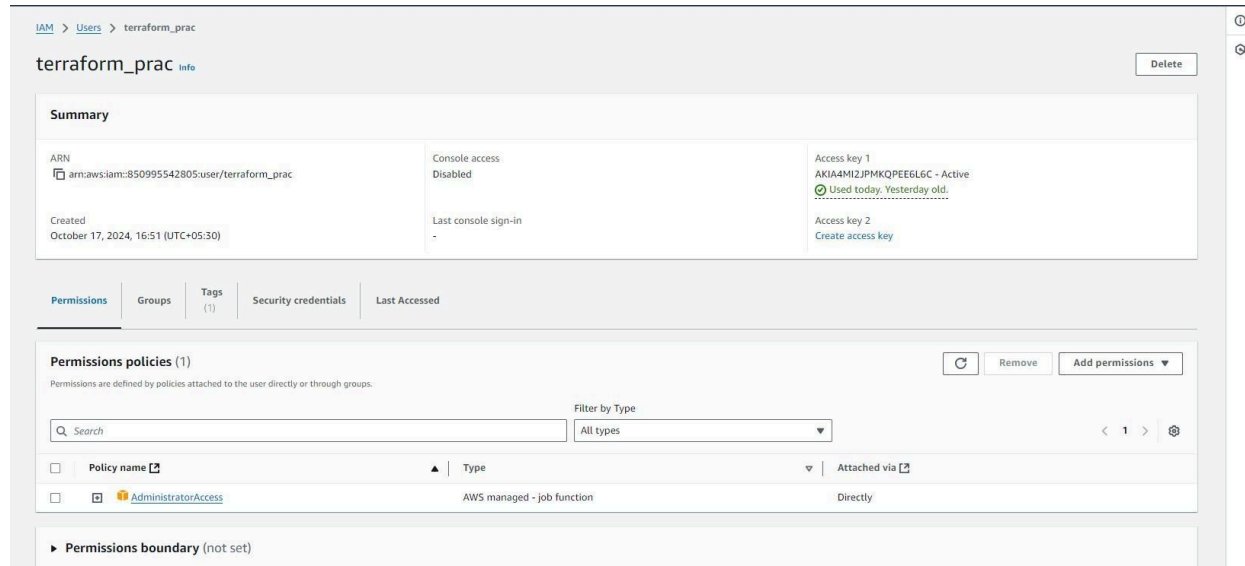
- 1) Install aws cli for your device from the following url <https://aws.amazon.com/cli/> . Complete all the steps and confirm this by checking its version in console by running the command aws



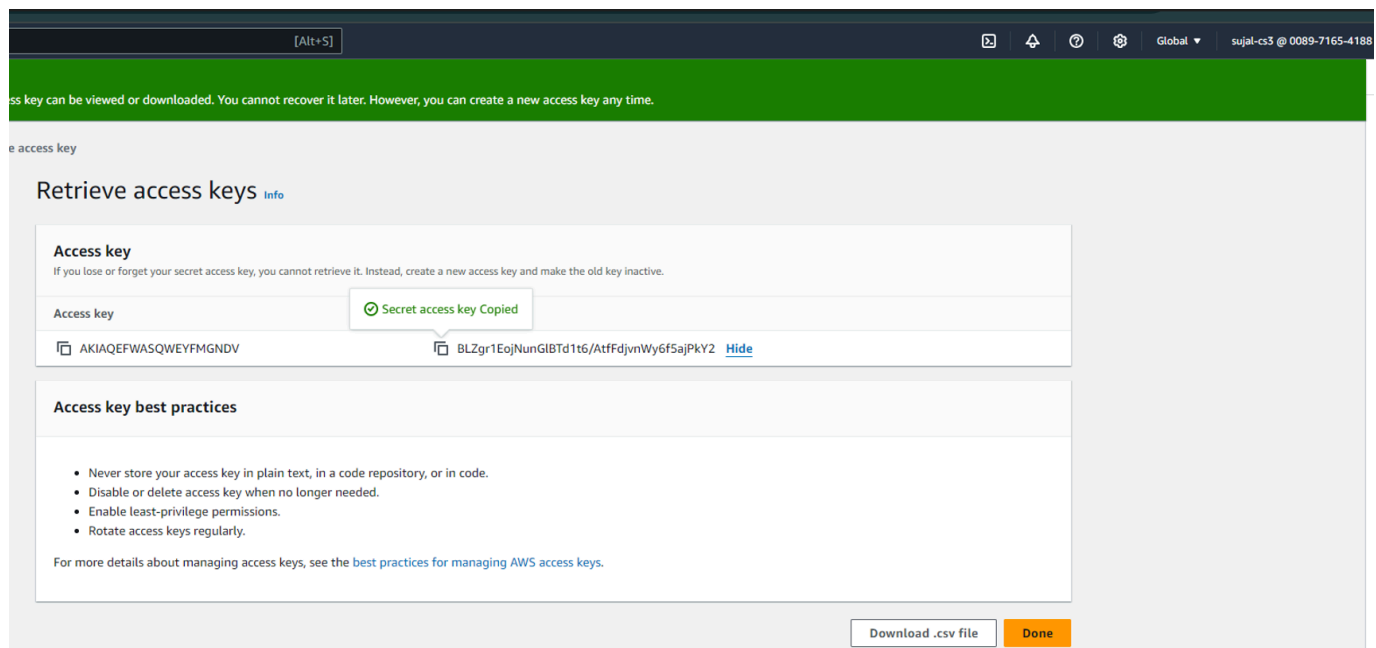
--version

```
C:\Users\ADMIN>aws --version
aws-cli/2.18.8 Python/3.12.6 Windows/11 exe/AMD64
```

2. Create a new IAM user . Give its permission as AdministratorAccess



3. Create a new Access key inside the IAM User by navigating to Security section inside your already created user . After creating the access key you will see something similar to this :-



4. Now create a new directory i have named it terraform-ec2-s3 in my desktop . Contents inside terraform-ec2-s3 folder :
- website(directory)
 - index.html
 - error.html
 - main.tf

The main.tf file inside this directory will hold our terraform code

```
C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3>touch main.tf
'touch' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3>echo. > main.tf

C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3>mkdir website
```

5. Initialize the terraform in the directory by running this command : terraform init

```
C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.72.1...
- Installed hashicorp/aws v5.72.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

7. Following are the contents for the files index.html , error.html and main.tf

a) For index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Static Website</title>
```

```
</head>
<body>
<h1>Welcome to Sujal's Static Website!</h1>
<p>This website is hosted on AWS S3.</p>
<p>This is Case study 3 which was performed by Sujal Tekwani</p>
</body>
</html>
```

b) For error.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Page Not Found</title>
<style>
body {
font-family: Arial, sans-serif; text-align: center;
padding: 50px;
}
h1 {
font-size: 50px;
}
p {
font-size: 20px;
}
a {
text-decoration: none; color: #007BFF;
}
</style>
</head>
<body>
```

```
<h1>Oops! Page Not Found</h1>
<p>We're sorry, but the page you were looking for doesn't exist.</p>
<p><a href="index.html">Go back to the homepage</a></p>
</body>
</html>
```

c) for main.tf

```
# AWS Provider
provider "aws" {
  region = "us-east-1" # Use the region you configured with aws configure
}

# IAM Role for EC2 to Access S3
resource "aws_iam_role" "ec2_role" {
  name = "ec2_role"

  assume_role_policy = jsonencode({
    Version: "2012-10-17",
    Statement: [{
      Action = "sts:AssumeRole",
      Effect = "Allow",
      Principal = {
        Service = "ec2.amazonaws.com"
      }
    }]
  })
}

# Attach S3 Full Access Policy to the EC2 Role
resource "aws_iam_policy_attachment" "ec2_s3_access" {
  name      = "ec2-s3-access"
  roles     = [aws_iam_role.ec2_role.name]
  policy_arn = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
}

# Instance Profile for EC2
resource "aws_iam_instance_profile" "ec2_profile" {
  name = "ec2_profile"
  role = aws_iam_role.ec2_role.name
}

# EC2 Instance
```

```
resource "aws_instance" "example" {
  ami          = "ami-06b21ccaeff8cd686" # Change this AMI based on your region if
necessary
  instance_type    = "t2.micro"
  iam_instance_profile = aws_iam_instance_profile.ec2_profile.name

  tags = {
    Name = "Terraform-EC2"
  }
}

# S3 Bucket for Static Website
resource "aws_s3_bucket" "website_bucket" {
  bucket = "bucket20112004sujaltekwni03" # Ensure this bucket name is unique globally
}

# Public Access Block for S3
resource "aws_s3_bucket_public_access_block" "website_bucket_public_access" {
  bucket          = aws_s3_bucket.website_bucket.id
  block_public_acls      = false
  ignore_public_acls    = false
  block_public_policy    = false
  restrict_public_buckets = false
}

# S3 Website Configuration
resource "aws_s3_bucket_website_configuration" "website_config" {
  bucket = aws_s3_bucket.website_bucket.id

  index_document {
    suffix = "index.html"
  }

  error_document {
    key = "error.html"
  }
}

# Upload files to the S3 bucket
resource "aws_s3_object" "website_index" {
  bucket = aws_s3_bucket.website_bucket.bucket
  key    = "index.html"
  source = "./website/index.html" # Adjust the path if necessary
}
```

```
}
```

```
resource "aws_s3_object" "website_error" {  
  bucket = aws_s3_bucket.website_bucket.bucket  
  key    = "error.html"  
  source = "../website/error.html" # Adjust the path if necessary  
}
```

8. Execute terraform.plan and terraform apply command and it should run successfully

```
C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3>terraform plan  
  
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:  
+ create  
  
Terraform will perform the following actions:  
  
# aws_iam_instance_profile.ec2_profile will be created  
+ resource "aws_iam_instance_profile" "ec2_profile" {  
  + arn          = (known after apply)  
  + create_date  = (known after apply)  
  + id          = (known after apply)  
  + name        = "ec2_profile"  
  + name_prefix  = (known after apply)  
  + path        = "/"  
  + role        = "ec2_role"  
  + tags_all     = (known after apply)  
  + unique_id    = (known after apply)  
}  
  
# aws_iam_policy_attachment.ec2_s3_access will be created  
+ resource "aws_iam_policy_attachment" "ec2_s3_access" {  
  + id          = (known after apply)  
  + name        = "ec2-s3-access"  
  + policy_arn  = "arn:aws:iam::aws:policy/AmazonS3FullAccess"  
  + roles      = [  
    + "ec2_role",  
  ]  
}  
  
# aws_iam_role.ec2_role will be created
```

```

# aws_instance.example will be created
+ resource "aws_instance" "example" {
  + ami              = "ami-06b21ccaef8cd686"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + cpu_core_count    = (known after apply)
  + cpu_threads_per_core = (known after apply)
  + disable_api_stop   = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized      = (known after apply)
  + get_password_data   = false
  + host_id             = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile = "ec2_profile"
  + id                 = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle = (known after apply)
  + instance_state      = (known after apply)
  + instance_type        = "t2.micro"
  + ipv6_address_count    = (known after apply)
  + ipv6_addresses        = (known after apply)
  + key_name              = (known after apply)
  + monitoring            = (known after apply)
  + outpost_arn          = (known after apply)
  + password_data        = (known after apply)
  + placement_group       = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns           = (known after apply)
  + private_ip            = (known after apply)
  + public_dns            = (known after apply)
  + public_ip            = (known after apply)
  + secondary_private_ips = (known after apply)
  + security_groups       = (known after apply)
  + source_dest_check     = true

```

```
C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3>terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```

# aws_iam_instance_profile.ec2_profile will be created
+ resource "aws_iam_instance_profile" "ec2_profile" {
  + arn              = (known after apply)
  + create_date      = (known after apply)
  + id              = (known after apply)
  + name            = "ec2_profile"
  + name_prefix      = (known after apply)
  + path            = "/"
  + role            = "ec2_role"
  + tags_all        = (known after apply)
  + unique_id       = (known after apply)
}

# aws_iam_policy_attachment.ec2_s3_access will be created
+ resource "aws_iam_policy_attachment" "ec2_s3_access" {
  + id              = (known after apply)
  + name            = "ec2-s3-access"
  + policy_arn      = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
  + roles           = [
    + "ec2_role",
  ]
}

# aws_iam_role.ec2_role will be created
+ resource "aws_iam_role" "ec2_role" {
  + arn              = (known after apply)
  + assume_role_policy = jsonencode(
    {
      + Statement = [
        + {
          + Action = "sts:AssumeRole"

```

```

+ id          = (known after apply)
+ key         = "index.html"
+ kms_key_id  = (known after apply)
+ server_side_encryption = (known after apply)
+ source      = "../website/index.html"
+ storage_class = (known after apply)
+ tags_all    = (known after apply)
+ version_id  = (known after apply)
}

Plan: 9 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

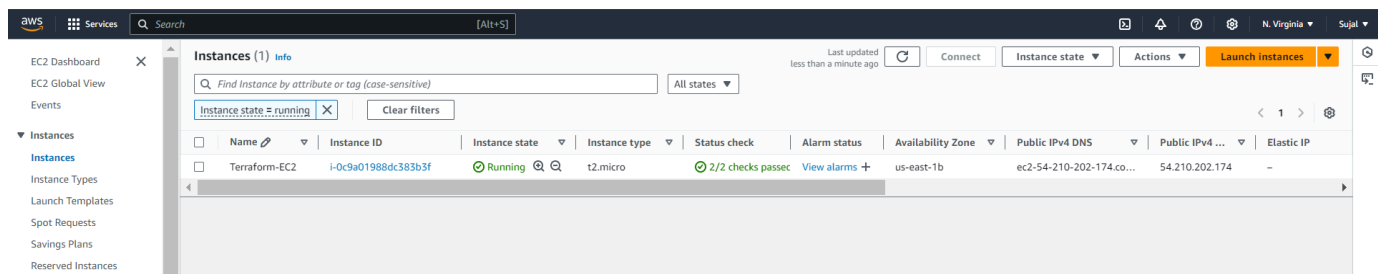
  Enter a value: yes

aws_iam_role.ec2_role: Creating...
aws_s3_bucket.website_bucket: Creating...
aws_iam_role.ec2_role: Creation complete after 2s [id=ec2_role]
aws_iam_policy_attachment.ec2_s3_access: Creating...
aws_iam_instance_profile.ec2_profile: Creating...
aws_iam_policy_attachment.ec2_s3_access: Creation complete after 0s [id=ec2-s3-access]
aws_iam_instance_profile.ec2_profile: Creation complete after 1s [id=ec2_profile]
aws_instance.example: Creating...
aws_s3_bucket.website_bucket: Creation complete after 5s [id=bucket20112004sujaltekwni03]
aws_s3_bucket_public_access_block.website_bucket_public_access: Creating...
aws_s3_bucket_website_configuration.website_config: Creating...
aws_s3_object.website_index: Creating...
aws_s3_object.website_error: Creating...
aws_s3_bucket_public_access_block.website_bucket_public_access: Creation complete after 2s [id=bucket20112004sujaltekwni03]
aws_s3_bucket_website_configuration.website_config: Creation complete after 2s [id=bucket20112004sujaltekwni03]
aws_s3_object.website_index: Creation complete after 2s [id=index.html]
aws_s3_object.website_error: Creation complete after 2s [id=error.html]
aws_instance.example: Still creating... [10s elapsed]
aws_instance.example: Still creating... [20s elapsed]
aws_instance.example: Creation complete after 22s [id=i-0c9a01988dc383b3f]

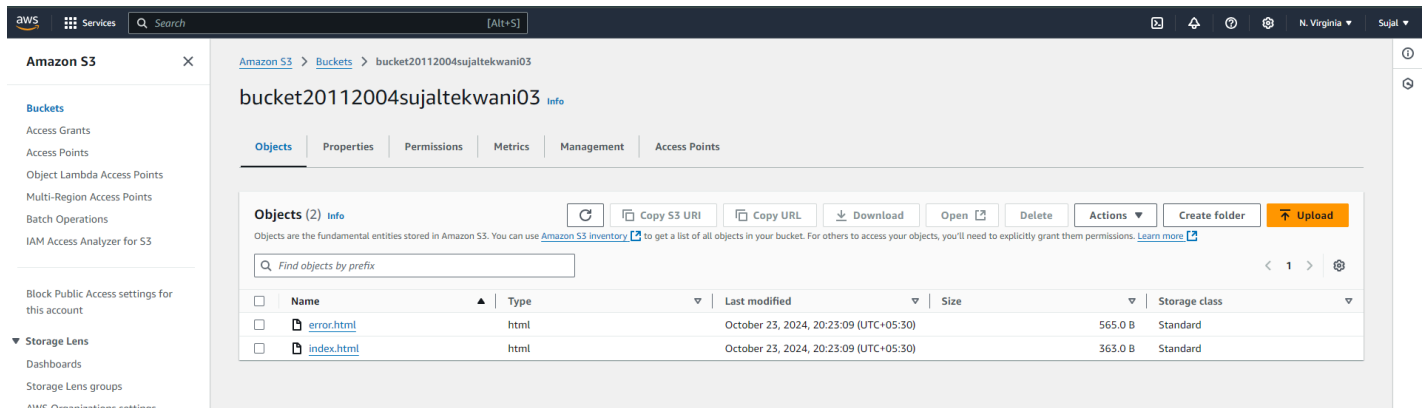
Apply complete! Resources: 9 added, 0 changed, 0 destroyed.

```

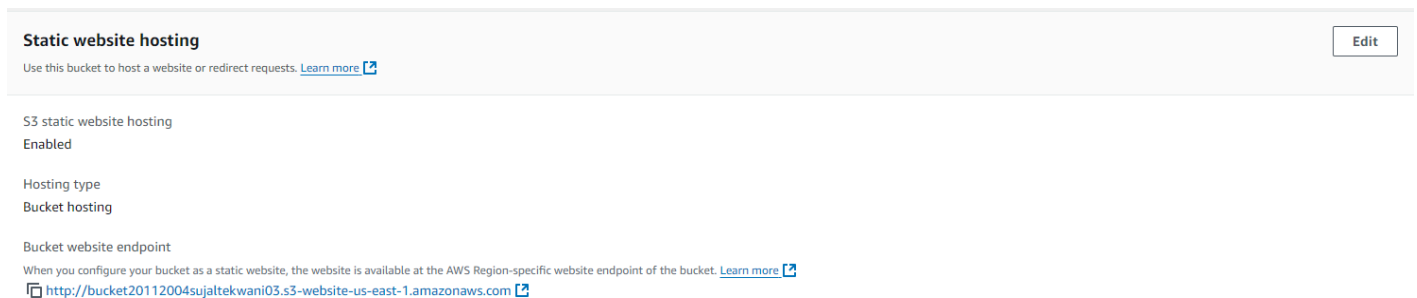
9. Since the terraform apply was executed successfully use should now be able to see a new ec2 instance “Terraform-EC2” and new s3 bucket named “bucket20112004sujaltekwni03” like the following screenshots



We can see in the bucket that error.html and index.html are successfully deployed through terraform



10. In the S3 bucket settings, confirm that the **Static website hosting** option is enabled.



11. Modify your terraform code in main.tf as follows :

```
# AWS Provider
```

```
provider "aws" {
```

```
    region = "us-east-1" # Use the region you configured with aws configure
```

```
}
```

```
# IAM Role for EC2 to Access S3
```

```
resource "aws_iam_role" "ec2_role" {
```

```
    name = "ec2_role"
```

```
    assume_role_policy = jsonencode({
```

```
        Version: "2012-10-17",
```

```
Statement: [{
  Action: "sts:AssumeRole",
  Effect: "Allow",
  Principal: {
    Service: "ec2.amazonaws.com"
  }
}]
})
}

# Attach S3 Full Access Policy to the EC2 Role
resource "aws_iam_policy_attachment" "ec2_s3_access" {
  name      = "ec2-s3-access"
  roles     = [aws_iam_role.ec2_role.name]
  policy_arn = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
}

# Instance Profile for EC2
resource "aws_iam_instance_profile" "ec2_profile" {
  name = "ec2_profile"
  role = aws_iam_role.ec2_role.name
}

# Generate an SSH Key Pair
resource "tls_private_key" "my_key" {
  algorithm = "RSA"
  rsa_bits  = 2048
}

resource "aws_key_pair" "my_key" {
  key_name  = "id_rsa_sujal63_public"
  public_key = tls_private_key.my_key.public_key_openssh
}
```

```
# Save the private key to a file (for local access)
```

```
resource "local_file" "private_key" {  
  filename = "${path.module}/id_rsa_private" # Adjust the path if necessary  
  content  = tls_private_key.my_key.private_key_pem  
}
```

```
# EC2 Instance
```

```
resource "aws_instance" "example" {  
  ami           = "ami-06b21ccaeff8cd686" # Change this AMI based on your region if  
necessary  
  instance_type    = "t2.micro"  
  iam_instance_profile = aws_iam_instance_profile.ec2_profile.name  
  key_name          = aws_key_pair.my_key.key_name # Associate the key pair with the instance  
  
  tags = {  
    Name = "Terraform-EC2"  
  }  
}
```

```
# S3 Bucket
```

```
resource "aws_s3_bucket" "website_bucket" {  
  bucket = "bucket20112004sujaltekwni03"  
}
```

```
resource "aws_s3_bucket_public_access_block" "website_bucket_public_access" {  
  bucket          = aws_s3_bucket.website_bucket.id  
  block_public_acls      = false  
  ignore_public_acls    = false  
  block_public_policy    = false  
  restrict_public_buckets = false  
}
```

```
resource "aws_s3_bucket_website_configuration" "website_config" {  
  bucket = aws_s3_bucket.website_bucket.id
```

```
index_document {
  suffix = "index.html"
}

error_document {
  key = "error.html"
}
}

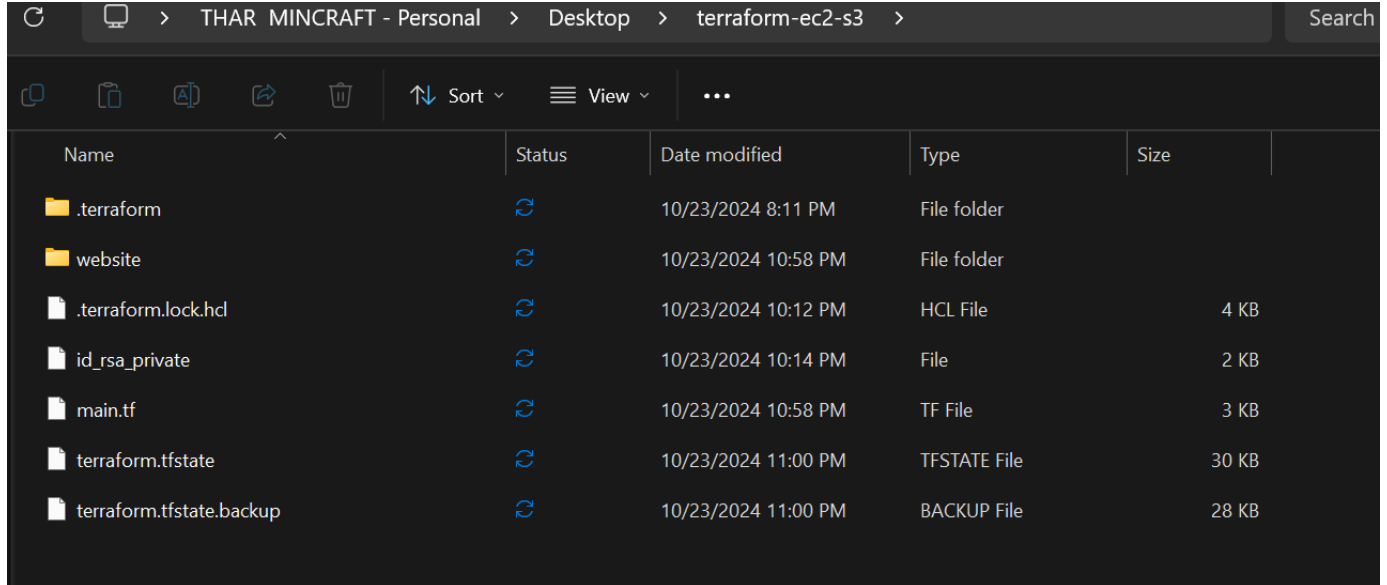
# Upload files to the S3 bucket
resource "aws_s3_object" "website_index" {
  bucket = aws_s3_bucket.website_bucket.bucket
  key    = "index.html"
  source = "./website/index.html"
}

resource "aws_s3_object" "website_error" {
  bucket = aws_s3_bucket.website_bucket.bucket
  key    = "error.html"
  source = "./website/error.html"
}
```

We have added public and private keys in the new code.

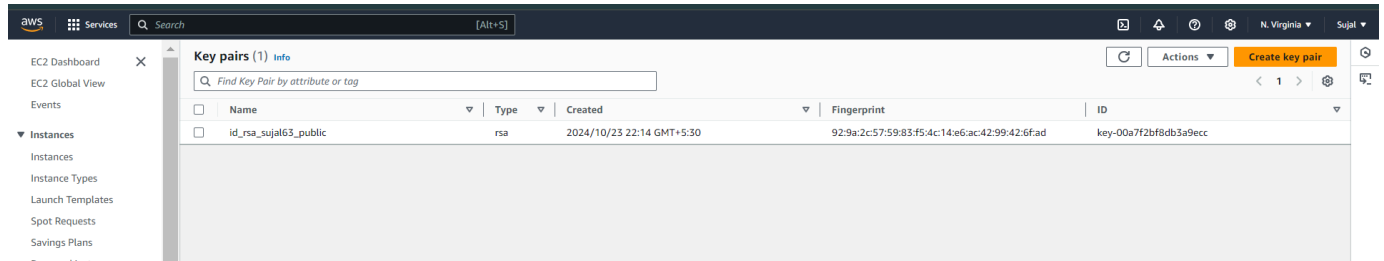
Now run terraform init , terraform plan , terraform apply again .

12. Now you can see the new key pair file named “id_rsa_private” has been downloaded in the same directory as all other files in the terraform-ec2-s3 directory



Name	Status	Date modified	Type	Size
.terraform		10/23/2024 8:11 PM	File folder	
website		10/23/2024 10:58 PM	File folder	
.terraform.lock.hcl		10/23/2024 10:12 PM	HCL File	4 KB
id_rsa_private		10/23/2024 10:14 PM	File	2 KB
main.tf		10/23/2024 10:58 PM	TF File	3 KB
terraform.tfstate		10/23/2024 11:00 PM	TFSTATE File	30 KB
terraform.tfstate.backup		10/23/2024 11:00 PM	BACKUP File	28 KB

We can also see the new public key name “id_rsa_sujal63_public” in aws key pairs section



Name	Type	Created	Fingerprint	ID
id_rsa_sujal63_public	rsa	2024/10/23 22:14 GMT+5:30	92:9a:2c:57:59:83:f5:4c:14:e6:ac:42:99:42:6f:ad	key-00a7f2bf8db3a9ecc

Use this command to remotely login into your ec2 instance

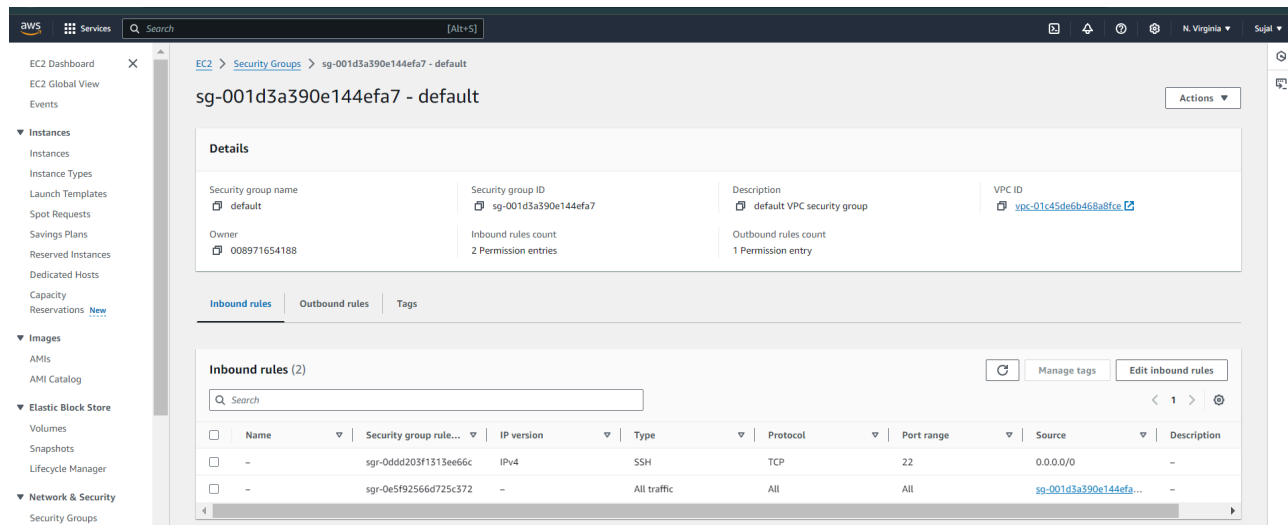
ssh -i "<path to your main.tf file/ has to be the same path>\id_rsa_private"

ec2-user@ec2-54-236-238-111.compute-1.amazonaws.com

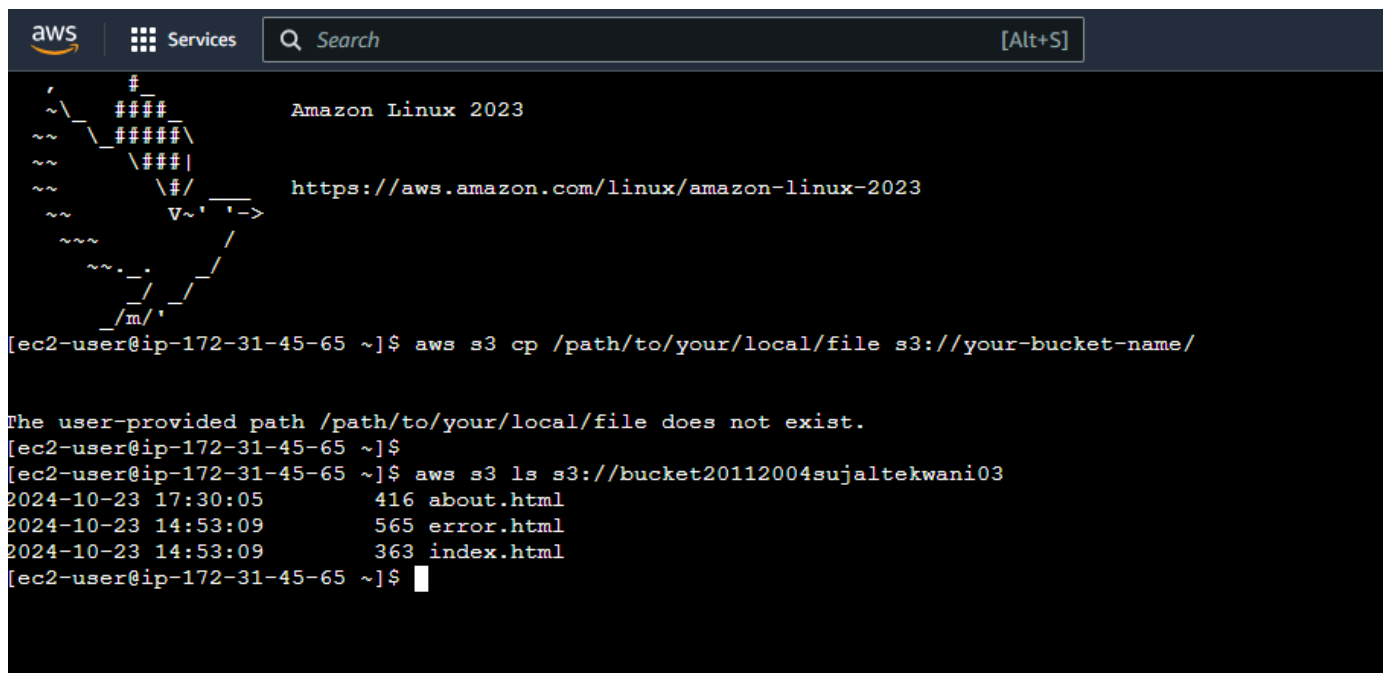
```
PS C:\Users\ADMIN> ssh -i "C:\Users\ADMIN\Desktop\terraform-ec2-s3\id_rsa_private" ec2-user@ec2-34-235-156-185.compute-1.amazonaws.com
ssh: connect to host ec2-34-235-156-185.compute-1.amazonaws.com port 22: Connection timed out
```

I encountered connection timeout issue this generally indicates that the connection request is being blocked, to resolve this

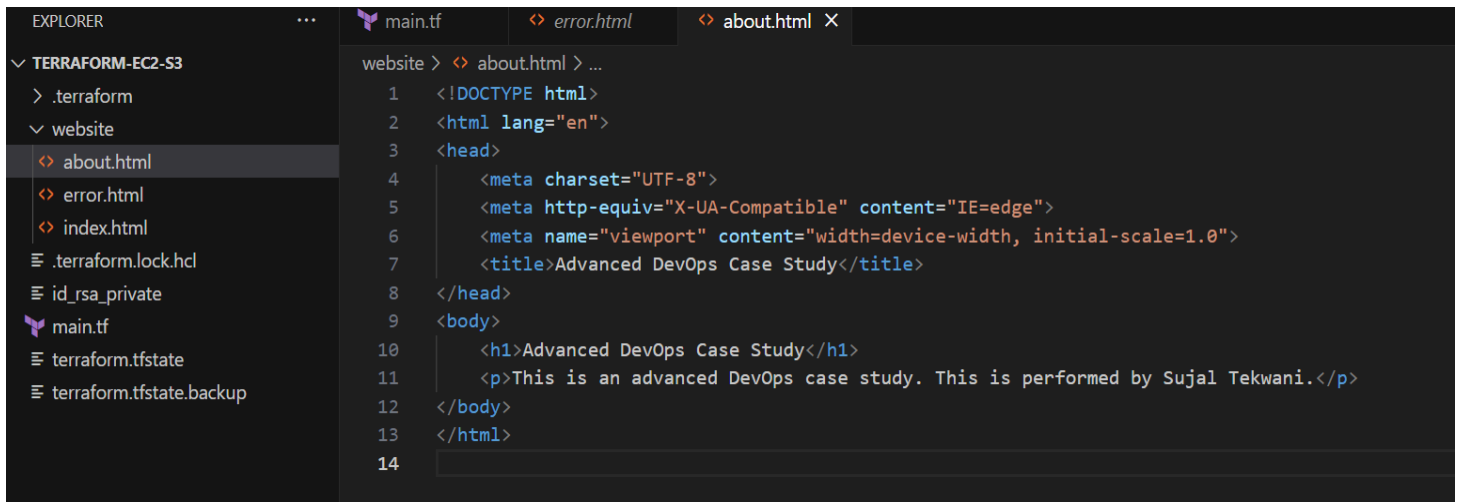
- Go to the AWS Management Console.
- Navigate to EC2 > Instances.
- Select your instance and find the Security Group in the description.
- Click on the security group link to go to the security group settings.
- Under the Inbound rules tab, ensure there is a rule that allows SSH



I have added SSH inbound rule in both the security groups . Now using the command of `ssh -i "C:\Users\ADMIN\Desktop\terraform-ec2-s3\id_rsa_private" ec2-user@ec2-34-235-156-185.compute-1.amazonaws.com` (in my case) remotely login into your ec2 instance



13. Now we will deploy another file called about.html in our bucket
Add the about.html in the website directory of the root directory terraform-ec2-s3



The screenshot shows the VS Code interface. On the left, the Explorer sidebar shows a project named 'TERRAFORM-EC2-S3' with a 'website' directory containing 'about.html', 'error.html', and 'index.html'. The main editor area shows the content of 'about.html' with the following HTML code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Advanced DevOps Case Study</title>
8 </head>
9 <body>
10  <h1>Advanced DevOps Case Study</h1>
11  <p>This is an advanced DevOps case study. This is performed by Sujal Tekwani.</p>
12 </body>
13 </html>
14
```

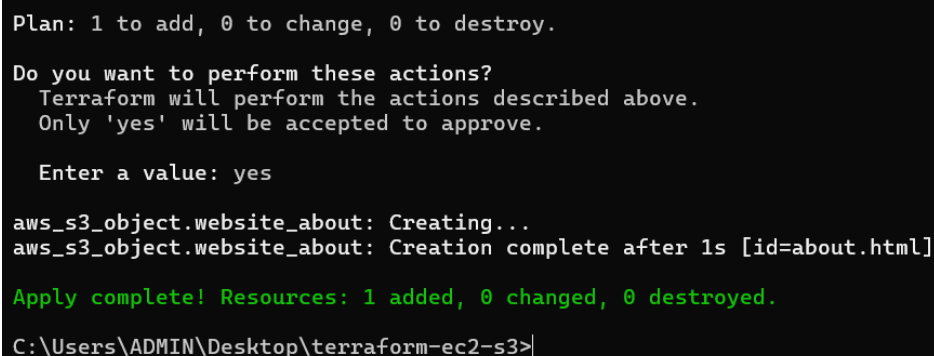
14. Add the following code at the end of main.tf

```
resource "aws_s3_object" "website_about" {
  bucket =
    aws_s3_bucket.website_bucket.bucket
  key    = "about.html"      # The name the file will have in
  source = "./website/about.html" # The local path to your
  file
}
```

This is a new aws_s3_object resource for the new file.

15. Now that we have added the about.html in website directory . Open the terminal and again execute the commands :

Terraform init , terraform plan and terraform apply to apply the changes



The screenshot shows the terminal output of the Terraform apply command. It displays the plan, a confirmation prompt, and the successful creation of the aws_s3_object resource.

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

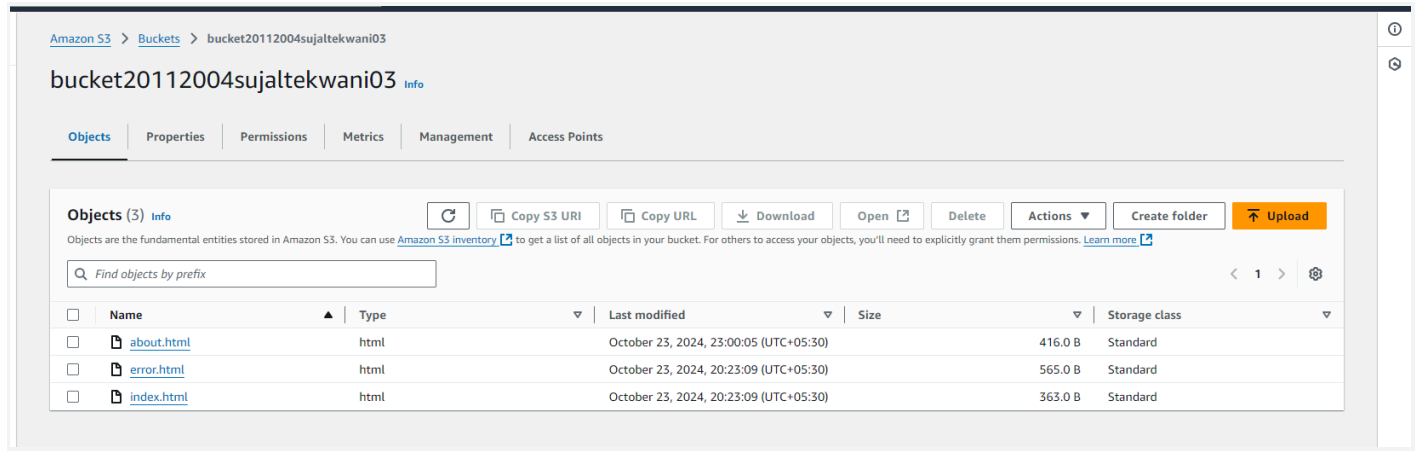
aws_s3_object.website_about: Creating...
aws_s3_object.website_about: Creation complete after 1s [id=about.html]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

C:\Users\ADMIN\Desktop\terraform-ec2-s3>
```

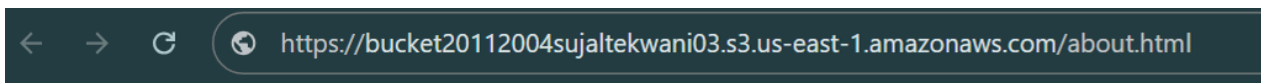
This screenshot of terraform apply says that the new resource (about.html) was successfully added in our bucket in aws called
bucket20112004sujaltekwni03

16. Check in the contents of your S3 bucket the new file should have been added



These are the files uploaded :

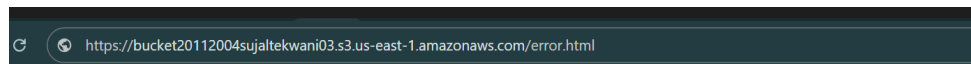
a) about.html



Advanced DevOps Case Study

This is an advanced DevOps case study. This is performed by Sujal Tekwani.

b) error.html

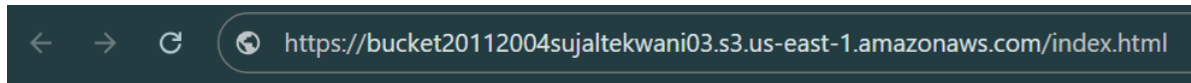


Oops! Page Not Found

We're sorry, but the page you were looking for doesn't exist.

[Go back to the homepage](#)

c) index.html



Welcome to Sujal's Static Website!

This website is hosted on AWS S3.

This is Case study 3 which was performed by Sujal Tekwani

Alternatively we can deploy static websites using ec2 also by following these steps :

17. Upload Files from Local Machine to EC2 using the command scp -i path/to/your/key.pem

```
scp -i C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3\id_rsa_private  
C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3\website\test.html  
ec2-user@3.94.117.185:/home/ec2-user/
```

```
Windows PowerShell  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows  
  
PS C:\Users\sujal> cd .\OneDrive\  
PS C:\Users\sujal\OneDrive> cd .\Desktop\  
PS C:\Users\sujal\OneDrive\Desktop> scp -i C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3\id_rsa_private C:\Users\sujal\OneDrive\Desktop\terraform-ec2-s3\website\test.html ec2-user@3.94.117.185:/home/ec2-user/  
The authenticity of host '3.94.117.185 (3.94.117.185)' can't be established.  
ED25519 key fingerprint is SHA256:dRTuJX2i+I8YBVaH1bb4x+0jPIkLGR9hCMJJLjp9qic.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])?  
Please type 'yes', 'no' or the fingerprint:  
Warning: Permanently added '3.94.117.185' (ED25519) to the list of known hosts.  
test.html 100% 415 1.8KB/s 00:00  
PS C:\Users\sujal\OneDrive\Desktop> |
```

18. In the console execute the following command to upload files to your S3 bucket:

```
aws s3 cp /home/ec2-user/test.html s3://bucket20112004sujaltekwni03/
```

```
[ec2-user@ip-172-31-45-65 ~]$ ^C
[ec2-user@ip-172-31-45-65 ~]$ aws s3 cp /home/ec2-user/test.html s3://bucket20112004sujaltekwni03/
upload: ./test.html to s3://bucket20112004sujaltekwni03/test.html
[ec2-user@ip-172-31-45-65 ~]$
```

i-09b93f8b0ad455a53 (Terraform-EC2)

By verifying using the ls command our file test.html was successfully uploaded to our s3 bucket

```
aws s3 ls s3://bucket20112004sujaltekwni03/
```

```
[ec2-user@ip-172-31-45-65 ~]$ ^C
[ec2-user@ip-172-31-45-65 ~]$ aws s3 cp /home/ec2-user/test.html s3://bucket20112004sujaltekwni03/
upload: ./test.html to s3://bucket20112004sujaltekwni03/test.html
[ec2-user@ip-172-31-45-65 ~]$ aws s3 ls s3://bucket20112004sujaltekwni03/
2024-10-23 17:30:05      416 about.html
2024-10-23 14:53:09      565 error.html
2024-10-23 14:53:09      363 index.html
2024-10-24 00:08:54      415 test.html
[ec2-user@ip-172-31-45-65 ~]$
```

i-09b93f8b0ad455a53 (Terraform-EC2)

Amazon S3 > Buckets > bucket20112004sujaltekwni03

bucket20112004sujaltekwni03 [Info](#)

[Objects](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [Access Points](#)

Objects (4) [Info](#)

[Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	about.html	html	October 23, 2024, 23:00:05 (UTC+05:30)	416.0 B	Standard
☐	error.html	html	October 23, 2024, 20:23:09 (UTC+05:30)	565.0 B	Standard
☐	index.html	html	October 23, 2024, 20:23:09 (UTC+05:30)	363.0 B	Standard
☐	test.html	html	October 24, 2024, 05:38:54 (UTC+05:30)	415.0 B	Standard

Conclusion

In this experiment, I used Terraform to automate the creation of AWS resources, including EC2 instances and an S3 bucket. After setting up the infrastructure, I deployed a sample HTML website by uploading the static files to the S3 bucket. By configuring the bucket for static website hosting, the HTML files were made publicly accessible. This process not only demonstrated how Terraform can automate cloud resource management but also how easily a simple website can be hosted on AWS using S3. The combination of automation and cloud hosting ensures a streamlined and efficient deployment process.